

AI-Agent Tool Descriptions and Naming

Describing Tools to the Agent

When developing an agentic AI system, one of the most critical aspects is ensuring that the agent understands the tools it has access to. In our previous tutorial, we explored how an AI agent interacts with an environment. Now, we extend that discussion to focus on tool definition, particularly the importance of naming, parameters, and structured metadata.

Example: Automating Documentation for Python Code

Imagine we are building an AI agent that scans through all Python files in a `src/` directory and automatically generates corresponding documentation files in a `docs/` directory. This agent will need to:

- List Python files in the `src/` directory.
- Read the content of each Python file.
- Write documentation files in the `docs/` directory.

Since file operations are straightforward for humans but ambiguous for an AI without context, we must clearly define these tools so the agent knows how to use them effectively.

Step 1: Defining a Tool with Structured Metadata

A basic tool definition in Python might look like this:

```
def list_python_files():  
    """Returns a list of all Python files in the src/ directory."""  
    return [f for f in os.listdir("src") if f.endswith(".py")]
```

This provides a function that retrieves all Python files in the `src/` directory, but for an AI system, we need a more structured way to describe it.

Step 2: Using JSON Schema to Define Parameters

When developers design APIs, they use structured documentation to describe available functions, their inputs, and their outputs. JSON Schema is a well-known format for defining APIs, making it a natural choice for AI agents as well. For example, a tool that reads a file should specify that it expects a `file_path` parameter of type `string`. JSON Schema allows us to express this in a standardized way:

```
{  
  "tool_name": "read_file",  
  "description": "Reads the content of a specified file.",  
  "parameters": {  
    "type": "object",  
    "required": [  
      "file_path": { "type": "string" }  
    ],  
    "properties": { "file_path": { "type": "string" } }  
  }  
}
```

Similarly, a tool for writing documentation should define that it requires a `file_name` and `content`:

```
{  
  "tool_name": "write_doc_file",  
  "description": "Writes a documentation file to the docs/ directory.",  
  "parameters": {  
    "type": "object",  
    "required": [  
      "file_name": { "type": "string" },  
      "content": { "type": "string" }  
    ],  
    "properties": { "file_name": "string" }  
  }  
}
```

By providing a JSON Schema for each tool:

- The AI can recognize the tool's purpose.
- The AI / Environment interface can validate input parameters before execution.

It may look strange that multiple parameters to a function are represented as an object. When we are getting the agent to output a tool / action selection, we are going to want it to output something like this:

```
{  
  "tool_name": "read_file",  
  "args": {  
    "file_path": "src/file.py"  
  }  
}
```

The schema describes the overall dictionary that will be used to capture the "args" to the function, so it is described as an object.

https://www.coursera.org/learn/ai-agents-python/ungradedWidget/RNyes/agent-tools-in-python

1/1